Compiler-Generated All-Reduce Compression (CGAC)

Per-Layer Algorithm Selection with PowerSGD Low-Rank, QSGD Quantization, and CFIP Native Passthrough in NeuralScript

Authors: Brandon Wiemer **Date:** April 2026 **Status:** Research Proposal

---

Abstract

We propose **CGAC (Compiler-Generated All-Reduce Compression)**, a compile-time gradient-compression technique for NSL distributed training that selects per-layer compression algorithms from a menu (PowerSGD, QSGD, signSGD, CFIP-passthrough, uncompressed) based on tensor shape, gradient statistics, and measured network bandwidth. Unlike PyTorch's PowerSGD communication hook which applies one algorithm globally and requires user-managed error-feedback buffers, CGAC's per-layer selection uses the right algorithm for each layer's gradient structure — rank-r PowerSGD for large matrix gradients, element-wise QSGD for small tensors, 1-bit signSGD for highly redundant layers, and CFIP format passthrough when the backward kernel already produces compressed gradients. CGAC integrates with FASE Deferred (reducing MHat/VHat's gradient contribution rather than a separate gradient buffer), CPDT's sharding decisions (compression respects tensor-parallel slicing), and CFIP (gradients in CFIP format reduced in that format, avoiding dequant-recompress). The compiler manages error-feedback buffers automatically; no user bookkeeping. Expected impact: 20-40% training throughput improvement on multi-node training at 10Gbps inter-node bandwidth; 5-15% improvement at faster 200Gbps networks; essentially free composition with CFIP where the backward already produces quantized gradients.

---

1. The All-Reduce Bottleneck

1.1 Where network bandwidth hurts

In data-parallel distributed training, after each backward pass the gradients are all-reduced across ranks: summed and broadcast. For a model with P parameters in N ranks connected by bandwidth B:

$$\text{all\_reduce\_time} = \frac{2 \cdot (N-1)}{N} \cdot \frac{P \cdot 4 \text{ bytes}}{B}$$

(Ring all-reduce; the $2(N-1)/N$ factor accounts for scatter + gather phases.)

For a 7B-parameter model with FP32 gradients (28 GB total), on a typical 10 Gbps Ethernet multi-node fabric:

- All-reduce time per step: $\frac{2 \cdot 7}{8} \cdot \frac{28\ \text{GB}}{1.25\ \text{GB/s}} \approx 40$ seconds for 8 ranks.

Compare to the backward pass compute time on an H100: approximately 1-2 seconds for this model. All-reduce is 20-40× the compute time at 10 Gbps. Even at 200 Gbps InfiniBand:

- $\frac{2 \cdot 7}{8} \cdot \frac{28\ \text{GB}}{25\ \text{GB/s}} \approx 2$ seconds — still comparable to backward compute.

At NVLink/NVSwitch intra-node bandwidth (600 Gbps): the all-reduce shrinks to ~300 ms and becomes a minor share. This is why single-node training has not motivated gradient compression in the NSL codebase to date — intra-node bandwidth hides the problem.

Multi-node is where compression pays off.

1.2 What research has established

The canonical techniques:

PowerSGD (Vogels et al., 2019). Approximates the gradient matrix with a low-rank factorization via power iteration. The factors P and Q are all-reduce-compatible (sum of approximations ≈ approximation of sum). Error feedback preserves convergence. 120× compression on LSTM, 54% communication time reduction on ResNet-18 CIFAR10. PyTorch has a built-in DDP communication hook. Critical caveat: a 2025 follow-up (PowerSGD+) identified a non-convergence counterexample in the original algorithm and proposes periodic SVD reprojection as a fix.

QSGD (Alistarh et al., 2017). Stochastic quantization of gradients: each element is probabilistically rounded to a quantization level. 4-8× compression with bounded variance increase. All-reduce-compatible when stochastic unbiased.

signSGD. 1-bit: only the sign of each gradient is transmitted. 32× compression. Not all-reduce-compatible (majority-vote aggregation is not associative); requires all-gather. Limits scaling.

TopK / DGC. Send only the top-k elements by magnitude. Up to 5000× compression but fails on small models. Not all-reduce-compatible.

1-bit Adam (Tang et al., 2021). Variance-normalized 1-bit gradients, Adam-specific integration.

LQ-SGD (2025). PowerSGD + logarithmic quantization of the factors. Adds 32/b compression on top of PowerSGD.

The trade-offs:

- Compression ratio.
- All-reduce compatibility (associativity).
- Convergence impact.
- Compression/decompression compute overhead.

No single method dominates. The right choice depends on network speed, batch size, model size, optimizer, and layer structure. One algorithm globally is a compromise.

1.3 Why NSL should have this

NSL has:

- **CPDT**: all-reduce primitive, process group topology, sharding decisions.
- **FASE**: gradient buffer management (Deferred mode eliminates the buffer).
- **CFIP**: backward kernels can produce gradients in compressed form.
- **WGGO**: per-layer decision-making infrastructure.

These compose naturally into a per-layer compression algorithm selector. CGAC is the missing integration.

---

2. Seven Compiler-Native Compression Innovations

2.1 Innovation 1: Per-layer algorithm selection

CGAC's WGGO pass reads each layer's gradient tensor shape and gradient statistics (measured during the first N training steps) and assigns one of:

Algorithm	Best for	Compression
PowerSGD rank-r	Large matrices with low-rank gradient structure	rank/(min_dim)
QSGD k-bit	Small-to-medium tensors, bounded variance desired	32/k
signSGD with EF	Highly redundant gradients (late-training)	32×
CFIP-passthrough	Gradients already in CFIP format	already compressed
Uncompressed	Tiny tensors, network fast enough	1×

Selection heuristics:

- **PowerSGD** for matrices where min(M, N) > 256 and measured gradient rank-ratio < 0.3 (most transformer linear layers).
- **QSGD** for 1D tensors (bias vectors, layer norms), small 2D tensors, and gradient tensors with no obvious low-rank structure.
- **signSGD** only during late-training phases with high redundancy; early-training convergence requires gradient magnitude information.
- **CFIP-passthrough** whenever the backward kernel emits gradients in CFIP format — avoid the dequant-and-recompress round-trip.
- **Uncompressed** for gradients with <1 MB total size (below the bandwidth-savings threshold).

The selection is made at compile time; the rank-ratio and gradient-stats measurement is an early-training profile pass.

2.2 Innovation 2: Compiler-managed error feedback

PowerSGD's error feedback (EF) requires a persistent per-tensor buffer that carries the previous step's compression residual forward. In PyTorch, the user manages this buffer; mistakes (not resetting between runs, aliasing, freeing early) are a common bug source.

In NSL, CGAC generates the EF buffer automatically:

- At model compile, CGAC allocates EF buffers for every PowerSGD-using layer.
- The buffer lifetime is bound to the training loop; M36's memory planner allocates and tracks it.
- EF state is checkpointed with model state (part of the optimizer checkpoint).
- Reset semantics are explicit: re-loading from checkpoint restores EF; starting fresh zeros EF.

QSGD-with-EF and signSGD-with-EF also use the same machinery. The compiler tracks which layers have EF enabled and emits the EF-update logic in the compressed gradient path.

2.3 Innovation 3: PowerSGD+ periodic reprojection by default

The 2025 PowerSGD+ paper establishes that the original PowerSGD has a non-convergence counterexample — there exist gradient distributions where PowerSGD fails to converge regardless of learning rate. PowerSGD+ adds periodic SVD reprojection of the low-rank subspace to escape this pathology.

CGAC uses PowerSGD+ as the default PowerSGD implementation. The reprojection runs every 100 steps (configurable), costs an SVD (a few milliseconds for typical ranks), and eliminates the non-convergence risk.

This is a small but principled difference from PyTorch's PowerSGD hook, which ships the original algorithm and relies on practice to avoid the pathology.

2.4 Innovation 4: CFIP format passthrough

When the backward kernel produces gradients already in a CFIP format (IL2, IM3), CGAC does not dequantize and recompress. Instead:

- The gradient is written to HBM in the CFIP format.
- CPDT's all-reduce uses a format-aware reduction op: element-wise sum in the CFIP format's representation (integer sum for the mantissa; per-block scale renormalization).
- The dequant happens once, on the receiving side, during the optimizer step.

The network bandwidth reduction is the format's compression factor (4-8× typical). No additional compression algorithm is needed; the CFIP format itself *is* the compression.

Block-scale renormalization handles the associativity issue. When summing 8 ranks' per-block CFIP contributions, the block's cumulative value may exceed the representable range; CGAC detects and adjusts the block scale on the receiving side. Most blocks stay within range; the overflow path is rare.

2.5 Innovation 5: Composition with FASE Deferred

When FASE is Deferred, there is no gradient buffer to reduce. Gradients are accumulated directly into the optimizer's MHat/VHat via the bias-corrected AdamW path.

CGAC's CPDT integration reads FASE's mode:

- FASE Immediate → compress the gradient buffer, reduce, dequant into buffer, optimizer reads buffer.
- FASE Deferred → compress MHat's gradient contribution (per step), reduce, dequant, accumulate into MHat.

The Deferred path is slightly more complex: the compressed entity is not the raw gradient but the gradient-divided-by-bias-correction-factor (which is what FASE accumulates into MHat). CGAC emits the correct compression target based on FASE mode.

This is the composition referenced in the composition paper's invariant I-17.

2.6 Innovation 6: Topology-aware bandwidth estimation

CPDT knows the process-group topology at runtime initialization. CGAC extends this with a bandwidth-estimation pass that runs once at initialization:

- Measure point-to-point bandwidth between pairs of ranks.
- Build a bandwidth matrix.
- Estimate effective ring-all-reduce bandwidth per process group.

The estimated bandwidth feeds CGAC's per-layer algorithm selection: if intra-node bandwidth is high (NVLink), uncompressed or lightly-compressed algorithms are preferred; if inter-node bandwidth is the bottleneck (Ethernet, InfiniBand), aggressive compression is preferred.

For a mixed-topology system (intra-node NVLink + inter-node Ethernet), CGAC can choose different algorithms for different process groups: uncompressed intra-node, PowerSGD inter-node. CPDT's hierarchical all-reduce primitive supports this.

2.7 Innovation 7: Layer-grouped all-reduce batching

Standard all-reduce treats each gradient tensor independently. For a 32-layer transformer, this means 32× the startup latency overhead (each all-reduce has a fixed startup cost).

CGAC groups layers into batches based on a shared compression algorithm and issues batched all-reduces:

- All layers using PowerSGD rank-r go into one all-reduce (concatenate the factors).
- All QSGD layers go into another (concatenate the quantized values).
- CFIP-passthrough gradients go into a third.

This reduces the total number of all-reduce operations and amortizes the per-op latency. On multi-node systems where per-op latency is significant, the batching improvement is 2-3× for small layers.

---

3. Algorithm Details

3.1 PowerSGD+ (CGAC's default rank-r algorithm)

Given gradient matrix $G \in \mathbb{R}^{M \times N}$ and rank r:

Forward step (compression):

1. Maintain $Q_t \in \mathbb{R}^{N \times r}$ (warm-start from previous step).
2. Compute $P_t = G_t \cdot Q_t \in \mathbb{R}^{M \times r}$.
3. All-reduce P_t across ranks (sum).
4. Orthonormalize P_t via QR.
5. Compute $Q_{t+1} = G_t^\top \cdot P_t \in \mathbb{R}^{N \times r}$.
6. All-reduce Q_{t+1} across ranks (sum).
7. Reconstructed gradient: $\hat{G}_t = P_t \cdot Q_{t+1}^\top$.
8. Error feedback: $\mathrm{EF}_t = G_t - \hat{G}_t$ (saved for next step).
9. Next step's gradient input: $G_{t+1} + \mathrm{EF}_t$.

Periodic reprojection (every 100 steps):

1. Compute SVD of the accumulated EF buffer.
2. Replace Q_t with the top-r right singular vectors.
3. Clear the accumulated EF.

Bandwidth: $2 \times r \cdot (M + N)$ floats per all-reduce, vs $M \cdot N$ for uncompressed. Compression ratio: $M \cdot N / (2r(M + N))$.

For $M = N = 4096$ and $r = 4$: compression ratio $\approx 1024\times$. (Though typical useful ranges are $r = 2$-32.)

3.2 QSGD with stochastic unbiased quantization

Given gradient vector $g \in \mathbb{R}^d$ and bit budget b (2, 4, 8):

Compression:

1. Compute $\|g\|_\infty$.
2. For each element g_i: compute quantization level $\ell_i \in \{0, 1, \ldots, 2^b - 1\}$ such that $\mathbb{E}[Q(g_i)] = g_i$ (stochastic rounding).
3. Transmit: scale $\|g\|_\infty$ + sign bits + quantization levels.

Aggregation: associative sum (integer addition of quantized levels).

Dequantization: $\hat{g}_i = (\ell_i / (2^b - 1)) \cdot \|g\|_\infty \cdot \text{sign}_i$.

Bandwidth: 1 scale + $d \cdot (b + 1)$ bits per tensor. Vs $d \cdot 32$ bits uncompressed: $32/(b + 1)\times$ compression.

For $b = 4$: 6.4× compression. For $b = 2$: 10.7× compression.

3.3 signSGD-with-EF

Given gradient g:

Compression:

1. Transmit only $\text{sign}(g)$ (1 bit per element).
2. Compute $|g|$ — residual for EF.

Aggregation: sum of signs (not associative in general; handled via all-gather + majority vote).

Dequantization: $\hat{g}_i = \text{sign}_i \cdot \alpha$ where α is a scalar scaling factor (all-reduced separately from the signs).

Bandwidth: 1 bit per element. 32× compression vs FP32.

CGAC uses signSGD only in late-training phases (last 10-20% of steps) where gradient magnitudes converge and the sign approximation is dominant anyway.

3.4 CFIP format passthrough

No algorithm per se — just reuse the format the gradient is already in.

Pseudocode:

```
if gradient.format == CFIP::IL2:
    # Gradient is 2-bit integer with per-block scale
    all_reduce_op = sum_with_block_rescale(IL2 format)
    all_reduce(gradient.data, op=all_reduce_op)
    # Block scales may need renormalization
    renormalize_blocks_that_overflowed()
elif gradient.format == CFIP::IM3:
    ...
```

Bandwidth: the CFIP format's compression (4-8×).

Associativity: block-rescale handling (rare overflow path).

---

4. Implementation Architecture

4.1 Pipeline position

CGAC operates in Stage 4 (WGGO) for algorithm selection and Stage 7 (runtime) for execution.

Stage 4 (WGGO):

1. Profile pass: first N steps measure per-tensor gradient statistics and rank structure.
2. Network bandwidth estimation: measure inter-rank bandwidth.
3. Per-layer algorithm selection: assign algorithm and hyperparameters (rank, bit budget) per layer.
4. Write decisions to WGGO decision table.

Stage 5-6 (lowering and codegen):

1. Emit compression/decompression kernels.
2. Allocate EF buffers (tracked by M36).

Stage 7 (runtime):

1. Per-step: backward produces gradients, each layer's gradient compressed by its selected algorithm.
2. Layers grouped into batches by algorithm.
3. CPDT executes batched all-reduces.
4. Per-step: decompression runs on received data; gradients delivered to the optimizer (or directly to MHat/VHat in FASE Deferred).

4.2 Kernel implementations

PowerSGD+ kernel:

- QR decomposition: cuSOLVER wrappers for small-r QR (r = 1, 2, 4, 8, 16, 32).
- Matmul P = G @ Q: cuBLAS call.
- Matmul Q_next = G^T @ P: cuBLAS call.
- SVD for periodic reprojection: cuSOLVER, runs every 100 steps.

QSGD kernel:

- Stochastic rounding: PTX emitted with PRNG state per warp.
- Absmax reduction: shuffle-based warp reduction + block reduction.
- Packing k-bit values: bitpacking PTX.

signSGD kernel:

- Sign extraction: trivial per-element PTX.
- Magnitude accumulation for EF: fused with sign extraction.
- Majority vote: all-gather + bitwise OR + popcount.

CFIP passthrough "kernel":

- Just the existing CFIP format; no new kernel.
- NCCL custom reduction op registered at init (supplied with the block-rescale aggregation function).

4.3 EF buffer allocation

M36's memory planner gains an EF allocation pool. EF buffers are:

- Allocated once at model compile time.
- Sized equal to the gradient tensor size (PowerSGD) or smaller (QSGD/signSGD EF only needs residual after quantization).
- Checkpointed with the optimizer state.
- Reset semantics: preserved across recovery; zeroed on explicit reset.

Total EF memory for a 7B model with PowerSGD rank-4 across all linear layers: approximately 30% of gradient memory (the fraction of parameters that are large linear layer weights). For a 7B model that is ~8 GB of EF buffers. Substantial but bounded.

4.4 Hierarchical all-reduce for mixed topology

When the process group contains multiple hierarchy levels (intra-node NVLink + inter-node Ethernet), CGAC issues a hierarchical all-reduce:

1. Intra-node all-reduce: uncompressed (fast NVLink), reduces within a node.
2. Inter-node all-reduce: compressed (PowerSGD or QSGD), reduces across nodes.
3. Intra-node broadcast: uncompressed, distributes result within node.

This two-phase approach lets each phase use the optimal algorithm for its bandwidth. The NCCL library supports this pattern directly via hierarchical collective configurations.

---

5. Integration with NSL Pipeline

5.1 New invariants

I-37 — CGAC algorithm per layer is compile-time fixed. Runtime cannot change the algorithm. Phase transitions (CTQS) can trigger re-compile with a new algorithm, but not mid-training algorithm swap.

I-38 — EF buffers persist across training steps. Restarting or checkpointing preserves EF state. Zeroing EF is an explicit user action.

I-39 — Block-rescale renormalization is rare and overflow-triggered. The compressed all-reduce's fast path does not renormalize; only blocks with detected overflow trigger the scale adjustment. This keeps the fast path fast.

I-40 — Hierarchical all-reduce respects network topology. When the process-group topology spans intra- and inter-node, CGAC uses a hierarchical reduction with appropriate algorithm per level.

I-41 — Layer-grouped batching preserves per-layer algorithm identity. Within a batch, all layers use the same algorithm. Batches across algorithms are separate all-reduces.

5.2 Failure modes

F-24 — PowerSGD non-convergence. Symptom: loss plateaus or diverges late in training. Root cause: the PowerSGD non-convergence pathology. Mitigation: PowerSGD+ periodic reprojection is the default (§2.3). If the user explicitly opts out of reprojection, a warning is emitted.

F-25 — EF buffer corruption. Symptom: gradient signal is wrong after recovery from checkpoint. Root cause: EF buffer not saved/restored with the rest of state. Mitigation: EF buffers are part of optimizer state checkpoint by default.

F-26 — Block-rescale overflow during all-reduce. Symptom: gradients have incorrect values for some parameters. Root cause: the rare overflow path wasn't triggered correctly. Mitigation: conservative overflow detection (trigger renormalization if any block exceeds 75% of representable range) + correctness test that verifies summed blocks never exceed range.

F-27 — Hierarchical all-reduce miscoordination. Symptom: some ranks see different gradients than others. Root cause: intra- and inter-node all-reduces not correctly coordinated. Mitigation: NCCL's hierarchical collective handles synchronization; CGAC relies on NCCL's correctness.

---

6. Evaluation Plan

6.1 Baselines

1. **Uncompressed all-reduce:** baseline convergence, baseline communication cost.
2. **PowerSGD rank-4 (PyTorch DDP hook):** industry-standard compressed baseline.
3. **QSGD 4-bit.**
4. **1-bit signSGD with EF.**
5. **CGAC (this work):** per-layer selection.
6. **CGAC + CFIP-passthrough:** when CFIP is also active.
7. **CGAC + FASE Deferred:** composition test.

6.2 Benchmarks

- **Multi-node training at 10 Gbps, 40 Gbps, 200 Gbps (simulated and real).**
- **Single-node NVLink (baseline; CGAC should provide minimal benefit here, confirming it correctly identifies the regime).**
- **Convergence parity:** training loss curves vs uncompressed baseline on small models (50M-400M params).
- **Throughput:** steps/second at various network bandwidths and model sizes.

6.3 Correctness tests

1. **Convergence parity:** training a small model (50M) for 5000 steps with CGAC must produce final loss within 1% of uncompressed baseline.
2. **EF buffer recovery:** checkpoint mid-training, restore, continue; final loss must match uncheckpointed run.
3. **PowerSGD+ reprojection correctness:** SVD-derived subspace must match empirical subspace to within numerical tolerance.
4. **Block-rescale invariant:** no summed block exceeds representable range after rescale.
5. **Hierarchical all-reduce correctness:** all ranks see identical final gradient values.

6.4 Implementation phases

Phase	Scope	Sessions
1	CGAC WGGO pass + algorithm selection	2
2	PowerSGD+ kernel (QR, SVD, matmul)	3
3	QSGD kernel (stochastic quant, bitpacking)	1
4	signSGD kernel (sign, EF, majority vote)	1
5	CFIP passthrough integration	2
6	EF buffer management via M36	1
7	Hierarchical all-reduce	2
8	FASE composition	1
9	Multi-node test infrastructure	2
10	Correctness and benchmarks	3
Total		**18**

Estimated scope: ~3200 LOC across ~8 files, 18 implementation sessions. Requires multi-node test environment (currently absent from NSL's test matrix).

---

7. Discussion

7.1 Why the multi-node dependency matters

CGAC's value proposition requires multi-node bandwidth constraints. On single-node NVLink systems, the uncompressed all-reduce is fast enough that compression's compute overhead can exceed its bandwidth savings. CGAC's per-layer selection correctly identifies this regime and prefers uncompressed — but without a multi-node test environment, this selection cannot be validated.

The recommendation in the extension-directions paper is to prioritize CGAC only after multi-node test infrastructure is available. Before then, CGAC's design can be stabilized against synthetic bandwidth-constrained tests, but end-to-end validation requires real multi-node.

7.2 Interaction with pipeline parallelism

CPDT's pipeline-parallel training has a separate communication pattern (point-to-point activation forwarding, not all-reduce). CGAC does not compress pipeline-parallel activations — they are on the hot path of the forward pass and compression overhead would hurt latency. Pipeline-parallel communication remains uncompressed.

7.3 Interaction with tensor parallelism

Tensor-parallel training does all-reduce *within* a layer (to synchronize partial matmul results across the TP group). These all-reduces are typically intra-node and very frequent. CGAC could compress them but the bandwidth savings are typically small (small tensors, high frequency) while the compute overhead is non-trivial. Default: uncompressed for TP; compressed only for data-parallel all-reduces. Configurable per-process-group.

7.4 Relation to PowerSGD+

CGAC's PowerSGD+ default is a direct application of the 2025 paper (Xie et al.). We do not claim novelty in the algorithm; we claim the integration: compile-time algorithm selection + compiler-managed EF + CFIP passthrough composition + FASE composition are the novel contributions.

---

8. Conclusion

CGAC provides NSL with compile-time per-layer selection of gradient compression algorithms for distributed training. The selection is driven by tensor shape, gradient statistics, and network bandwidth; the supported algorithms include PowerSGD+ (with default periodic reprojection), QSGD, signSGD, CFIP-format passthrough, and uncompressed. The compiler manages error-feedback buffers, integrates with FASE's gradient accumulation mode, and emits hierarchical all-reduces on mixed-topology systems. Expected impact is 20-40% training throughput improvement on 10 Gbps multi-node training, with graceful degradation at higher bandwidths where compression is less valuable. The technique is gated on a multi-node test environment which NSL does not yet have; once that is in place, CGAC becomes a Tier-1 priority.

References

1. Vogels, T. et al. (2019). *PowerSGD: Practical Low-Rank Gradient Compression for Distributed Optimization*. NeurIPS 2019.
2. Alistarh, D. et al. (2017). *QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding*. NeurIPS 2017.
3. Bernstein, J. et al. (2018). *signSGD: Compressed Optimisation for Non-Convex Problems*. ICML 2018.
4. Tang, H. et al. (2021). *1-bit Adam: Communication Efficient Large-Scale Training with Adam's Convergence Speed*. ICML 2021.
5. Seide, F. et al. (2014). *1-Bit Stochastic Gradient Descent and Its Application to Data-Parallel Distributed Training of Speech DNNs*. INTERSPEECH 2014.
6. Karimireddy, S. P. et al. (2019). *Error Feedback Fixes SignSGD and Other Gradient Compression Schemes*. ICML 2019.
7. Xie, S. et al. (2025). *From PowerSGD to PowerSGD+: Low-Rank Gradient Compression for Distributed Optimization with Convergence Guarantees*. arXiv:2509.11254.
8. Aragani, V. M. et al. (2025). *Efficient Distributed Training through Gradient Compression with Sparsification and Quantization Techniques*. International Journal of Inventions in Engineering & Science Technology.
9. Agarwal, S. et al. (2022). *On the Utility of Gradient Compression in Distributed Training Systems*. MLSys 2022.
10. LQ-SGD Authors. (2025). *Trustworthy Efficient Communication for Distributed Learning using LQ-SGD Algorithm*. arXiv:2506.17974.